工作流 Python 代码节点沙箱与云函数

01. Python 代码沙箱环境

目前在 LLMOps 项目中，工作流的 `Code节点` 目前会将前端用户传递的代码在 LLMOps 的环境中执行，如果用户传递了恶意代码，在项目中不经过任何处理直接执行，会导致项目源码、数据泄露、环境被破坏等问题。

针对这类 `云端代码执行` 的场景，一般来说，会使用 `代码沙箱` 来隔离其运行环境，简单理解 `沙箱`，即 sandbox，顾名思义，就是让你的程序跑在一个隔离的环境下，不对外界的其他程序造成影响，通过创建类似沙盒的独立作业环境，在其内部运行的程序并不能对硬盘产生永久性的影响。

作为程序员最熟悉的 `leetcode`，当我们在提交算法题代码时，会用到沙箱创建一个独立的运行环境运行这段代码，保证机器的安全性；在服务器中使用 docker 创建一个独立的应用容器；与我们最相近的就是我们的浏览器窗口，每个浏览器窗口其实也是一个沙箱。

从零开始开发一个 `Python代码沙箱` 是一项非常庞大且复杂的任务，需要考虑 `禁用指定模块`、`禁止联网`、`限制内存使用`、`限制超时时间`、`限制底层硬件访问`、`并发执行代码`、`防止沙箱逃逸` 等，所以在课时中，我们可以使用一些相对成熟的开源 `Python代码沙箱` 方案。

Python 沙箱项目推荐：

- Terrarium：https://github.com/cohere-ai/cohere-terrarium
- Dify：https://github.com/langgenius/dify-sandbox

这里我们就以 Dify Sandbox 为例，Dify Sanbox 使用 Docker 容器进行设计，它需要以下依赖项：

- libseccomp：一个用于系统调用（syscall）过滤的库，主要用于增强安全性。它可以限制应用程序可以调用的系统调用，从而减少攻击面，常用于容器（如 Docker）或沙箱环境中。
- pkg-config：一个用于管理编译时库依赖的工具。它帮助编译器找到库的头文件和链接信息，通常用于 gcc 或 clang 编译程序时自动处理依赖项。
- gcc（GNU Compiler Collection）：GNU 编译器集合，支持多种编程语言（如 C、C++、Go、Fortran 及其他），是 Linux 和类 Unix 系统上最常用的 C/C++ 编译器之一。
- golang 1.20.6：Go 语言（Golang）1.20.6 版本。

Dify Sanbox 采用了 `seccomp` 机制进行系统调用过滤，防止恶意代码执行不安全操作，其底层的运行逻辑也非常简单，流程拆解如下：

1. `cmd/lib/python/main.go` 通过 `Go Build` 生成 `python.so`，这是一个动态库，包含 `DifySeccomp` 方法，创建安全的执行环境。
2. `internal/server/server.go` 通过 `Go Run` 启动服务器，处理从 HTTP 请求传递的 Python 代码，并管理代码的安全执行。

并且 Dify 提供了一个可以直接部署的 Docker 镜像，集成在 docker-compose 中也非常简单，示例如下：

```yaml
version: '3'
services:
  sandbox:
    image: langgenius/dify-sandbox:0.1.0
    restart: always
    cap_add:
      - SYS_ADMIN
    environment:
      API_KEY: dify-sandbox
      GIN_MODE: release
      WORKER_TIMEOUT: 15
```

容器默认暴露的端口号是 `8194`，启动容器后即可通过 `HTTP请求` 访问该容器，传递对应信息并使用 `POST` 方法即可访问该容器：

1. `X-Api-Key`：headers 参数，容器配置的 API_KEY，用于确保访问的安全。
2. `language`：request_body 参数，需要执行的编程语言，支持 `python3` 和 `nodejs` 两种。
3. `preload`：request_body 参数，预加载代码字符串，例如代码中需要 `import` 加载的库。
4. `code`：request_body 参数，需要执行的代码字符串，支持 `python` 和 `nodejs` 两种代码。

示例：

```
POST: http://localhost:8194/v1/sandbox/run

headers: {"X-Api-Key": "dify-sandbox"}

request_body: {
    "language": "python3",
    "preload": "import json\nimport datetime\nimport math\nimport random\nimport re\nimport string\nimport sys\nimport time\nimport traceback\nimport uuid\nimport os\nimport base64\nimport hashlib\nimport hmac\nimport binascii\nimport collections\nimport functools\nimport operator\nimport itertools",
    "code": "这里实现需要执行的代码，并且将需要返回的内容使用print打印出来即可"
}
```

接口返回的数据结构如下：

```
{
    "code": "int",  // 状态码
    "message": "str",  // 接口提示消息
    "data": {
        "stdout": "str",  // 执行输出结果
        "error": "str"  // 错误信息
    }
}
```

这样在 LLMOps 项目中，如果想接入 `Dify Sandbox`，总共有 2 步：

1. 将 `代码节点` 的外层函数去除，并预设填充 `params` 变量，将需要返回的数据使用 `print` 函数进行打印。
2. 将调整后的代码按照上述的 HTTP 请求规则，发送请求后获取 `stdout` 即可。

02. 腾讯云函数执行代码

上述的示例，我们分享的是本地部署 `SandBox` 沙箱，除了这种方案之外，还可以使用云服务提供商的 `云函数` 来执行对应的代码，每个云函数运行实例本身就相互隔离，甚至可以设置网络环境、可运行使用的内存大小、超时限制等。

链接：https://cloud.tencent.com/product/scf

实现思路也很简单：

1. 在云函数后台创建 `自定义函数`，并选择运行环境为 `Python 3.10`；
2. 触发方式选择 `API网关`，这样可以通过 HTTP 请求执行代码；
3. 在代码编辑页面，编写 `index.py`，如下：

```
import json

def main_handler(event, context):
    try:
        body = json.loads(event.get("body", "{}"))
        code = body.get("code", "")
        func_name = body.get("func_name", "")
        args = body.get("args", [])
        kwargs = body.get("kwargs", {})

        exec_globals = {}
        exec(code, {"__builtins__": {}}, exec_globals)

        if func_name in exec_globals and callable(exec_globals[func_name]):
            result = exec_globals[func_name](*args, **kwargs)
            return {
                "statusCode": 200,
                "body": json.dumps({"result": result})
            }
        else:
            return {
                "statusCode": 400,
                "body": json.dumps({"error": "Function not found or not
callable"})
            }

    except Exception as e:
        return {
            "statusCode": 500,
            "body": json.dumps({"error": str(e)})
        }
```

函数的含义也很简单，接收从 HTTP 请求传递的参数信息，例如 `code(代码)`、`args/kwargs(函数参数)`、`func_name(函数名字)`，在内部组件拼接函数，并使用 `exec` 执行函数得到对应数据，将结果拷贝到 `exec_globals` 后返回。

这样如果需要执行某个函数，只需要 post 传递类似如下数据即可：

```
{
  "code": "def add(x, y): return x + y",
  "func_name": "add",
  "args": [5, 10]
}
```

输出结果：

```
{
    "result": 15
}
```

□